Column-Level Security

Apache Spark Query Compiler Extension

Design Document & Incident Reports

Column-Level Security in Apache Spark via Query Compiler Extension

Overview

This document describes the design and implementation of a Column-Level Security (CLS) framework for Apache Spark, enforced at the query compiler level. The system integrates with Apache Ranger and Hive Metastore (HMS) to apply fine-grained column access policies without requiring changes to underlying data or storage layers.

Architecture

Policy Propagation

Column-level security policies are defined and managed in Apache Ranger. These policies are propagated to the query engine via a Ranger Proxy, which acts as a bridge between Ranger's policy store and the Spark query compiler. The proxy translates Ranger column-level permissions into a set of allowed columns per user per relation, stored as HMS table properties.

```
Ranger Policy Store ↓ Ranger Proxy ↓ Hive Metastore (HMS) ← secure column metadata stored as table properties ↓ Spark Extension (Query Compiler) ↓ Secure Projection applied at plan construction time
```

Enforcement Model

Tables

For regular tables (Parquet, Delta, Iceberg, Hudi etc.), the CLS framework intercepts catalog relation resolution and applies a Secure Projection directly over the scanned relation. This restricts the set of attributes visible to the query optimizer and downstream execution — columns the current user is not permitted to access are physically absent from the logical plan.

```
RelationV2[order_id, cls_customer_id, amount, cls_order_date] ↓ Secure Projection applied
Project[cls_customer_id, cls_order_date] +- RelationV2[order_id, cls_customer_id, amount,
cls_order_date]
```

Views — Policy Isolation Principle

For views, a deliberate and important design decision is made:

> *Only the view-level column policy applies. The underlying table's column policy is irrelevant.*

This means a user's access to columns through a view is governed entirely by what the view owner has declared and what Ranger has permitted at the view boundary — not by what the underlying table permits. This is consistent with standard SQL view semantics, where a view acts as a security boundary. This design enables:

- Exposing a restricted subset of a sensitive table through a view, without requiring the consumer to have any direct table permissions
- Defining view-level column masks or filters that differ from table-level policies
- Simplifying permission management by decoupling view consumers from table owners

Implementation: Spark Extension

The CLS enforcement is implemented as a Spark SQL Extension using the SparkSessionExtensions API. The extension hooks into Spark's query compilation pipeline at two points.

1. Relation Resolution (injectResolutionRule)

A custom resolution rule (ResolveCatalogViews) intercepts catalog relation lookups. When a view is resolved, the view SQL text is parsed and analyzed internally. A CustomView logical plan node wraps the analyzed view plan, exposing only the permitted columns as its output (secureOutput). The underlying full plan is stored internally as member — invisible to the outer query's analyzer.

```
case class CustomView( desc: CatalogTable, member: LogicalPlan, // full internal plan — not visible to outer query secureOutput: Seq[Attribute] // only permitted columns — what outer query sees ) extends LeafNode
```

CustomView extends LeafNode deliberately — this prevents Spark's optimizer rules (such as ColumnPruning and PushPredicateThroughProject) from descending into the internal plan and bypassing the security boundary.

2. Post-Hoc Resolution (injectPostHocResolutionRule)

A ViewSecurityRule runs after the main analysis phase in two passes:

Pass 1 — Access Enforcement: Before substituting the CustomView, the rule inspects all expressions in nodes whose direct child is a CustomView. Any reference to a column that is declared in the view schema (desc.schema) but not in secureOutput results in an AnalysisException being thrown. Restricted column names are derived from desc.schema rather than member.output — this excludes engine-internal hidden columns (Delta transaction metadata, Iceberg file columns etc.) from the security check, preventing false positives on system columns.

```
User query: WHERE order_id IS NOT NULL order_id declared in view schema ✓ order_id NOT in secureOutput ✓ → AnalysisException: Access denied: column 'order_id' in view 'v_ptbl'
```

Pass 2 — Substitution: After enforcement passes, CustomView is substituted with its member plan. Spark's analyzer re-runs post substitution, naturally inlining CTEs and resolving any remaining references against the now-visible internal plan.

Security Guarantees

Access Pattern	Enforcement
SELECT restricted_col FROM view	Blocked — not in CustomView.output, AnalysisException at resolution
WHERE restricted_col = x	Blocked — caught by Pass 1 of ViewSecurityRule
HAVING agg > (SELECT COUNT(*) FROM view WHERE restricted_col IS NOT NULL)	Blocked — transformUpWithSubqueries reaches scalar subquery
JOIN ON t.restricted_col = view.restricted_col	Blocked — caught in join condition expressions
SELECT allowed_col FROM view	Permitted — resolved normally, executes cleanly
Delta/Iceberg hidden columns (_commit_timestamp, _file etc.)	Not affected — enforcement scoped to desc.schema declared columns only

Key Design Decisions

LeafNode over UnaryNode

Making CustomView a LeafNode ensures the optimizer cannot traverse into member and push projections or predicates through the security boundary. A UnaryNode would expose member as a child, allowing optimizer rules to violate the CLS contract.

desc.schema for Restriction Scope

Restricted column names are derived from the view's declared schema (CatalogTable.schema) rather than member.output. This excludes engine-internal hidden columns from the security check, preventing false positives on system columns that users legitimately reference.

View Policy Isolation

The CustomView node is constructed using only the view-level Ranger policy. The underlying table's column ACLs are not consulted when resolving view access — the view boundary is the sole enforcement point for view consumers.

Post-Hoc Enforcement Timing

Enforcement runs in injectPostHocResolutionRule after the main analyzer pass but before optimizer rules. At this point, restricted column references are still UnresolvedAttribute nodes (since CustomView.output does not expose them), making interception clean and unambiguous.

Incident Report 1: Schema Synchronisation Hotfix

Incident Summary

In the first production drop of the CLS framework, a critical gap was identified for customer NAVI: schema synchronisation was not implemented for the specific case where a table is created on an existing Delta log (i.e., the Delta log already exists on object storage but the HMS entry is being created fresh or recreated). All other schema mutation and evaluation paths were correctly covered in the initial release build. This was an isolated miss affecting only the CREATE TABLE on existing Delta log scenario, causing a divergence between what HMS believed the table schema to be (the source of truth for CLS permitted columns) and what the Delta log actually contained — resulting in incorrect secure projections being applied at query time.

Root Cause

The CLS enforcement model designates HMS as the single source of truth for column-level security metadata. Ranger-propagated column policies are stored as HMS table properties and the secure projection applied at query compile time is derived entirely from the HMS schema.

The initial release build correctly handled schema synchronisation across all known write and evaluation paths. The single missed case was CREATE TABLE over an existing Delta log — a common pattern in data lake environments where the storage layer outlives the catalog entry. In this scenario:

```
Object Storage: Delta log exists with schema [A, B, C, D] HMS Entry: Created fresh / recreated with schema [A, B] ← stale or partial ↓ CLS reads HMS schema → secureOutput based on [A, B] Delta log serves → actual data has [A, B, C, D] ↓ Columns C and D: neither secured nor visible Columns A and B: projected correctly but HMS schema diverged from reality
```

The secure projection was being applied against a schema that did not reflect the actual Delta log state, making the security boundary unreliable.

Fix

Immediate Hotfix (NAVI)

Schema synchronisation was introduced specifically for the CREATE TABLE on existing Delta log path. When a CREATE TABLE statement is executed and the target location already contains a Delta log, the HMS schema is now reconciled against the Delta log schema before CLS metadata is written — bringing this path in line with all other schema paths already covered in the release build.

The table below shows the complete coverage picture across all schema lifecycle paths:

Path	Release Build	Hotfix
CREATE TABLE on existing Delta log	Not Included	Fixed
CREATE OR REPLACE TABLE	Included	—
ALTER TABLE schema evolution	Included	—
Schema-on-read evaluation at query time	Included	—
REPLACE WHERE / partition overwrite	Included	—
Delta log schema merge (mergeSchema=true)	Included	—

Architectural Principle Established

HMS schema is the CLS source of truth. Any operation that can mutate or evaluate a table schema — whether at DDL time, write time, or read time — must synchronise the HMS schema against the Delta log before CLS metadata is read or written. This ensures that the secure projection applied at query compile time always reflects the true physical schema of the underlying data, and that no column can become silently inaccessible or incorrectly exposed due to schema drift between HMS and the Delta log.

Incident Report 2: DeltaMerge Source Corruption via CLS Star Expansion

Incident Summary

During the application of CLS secure projections, a corruption was introduced in DeltaMerge operations where the SET clause contained a wildcard (*) expansion. The CLS implementation correctly handled * expansion in FROM clauses but did not account for * in the SET clause of a MERGE statement. This gap was not covered by the test suite at the time of release.

Root Cause

The CLS framework applies secure projections by intercepting column references and restricting them to the permitted set. When a SELECT * appears in a FROM clause, the implementation correctly expands and filters the wildcard against the secure column list.

However, in a MERGE INTO statement of the form:

```
MERGE INTO target t USING source_view s ON t.id = s.id WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *
```

The SET * and INSERT * wildcards in the WHEN MATCHED and WHEN NOT MATCHED clauses represent a different syntactic and semantic expansion path from SELECT * in a FROM clause. Internally, Delta's DeltaMerge logical plan node resolves * in SET operations differently — it expands against the source relation's output, not the target's schema.

The CLS implementation:

- Applied secure projection to the source view (correctly restricting columns visible from the view)
- Did not account for SET * / INSERT * expansion in the DeltaMerge node, which was expanding against the pre-CLS source schema

This caused the SET * to resolve against a corrupted or inconsistent column mapping — either including restricted columns that should have been excluded, or producing mismatched column assignments between source and target.

```
MERGE source = view (order_id, cls_customer_id, amount, cls_order_date) CLS secure projection applied to FROM clause: source visible to FROM = [cls_customer_id, cls_order_date] ✓ SET * expands against: DeltaMerge internal source = [order_id, cls_customer_id, amount, cls_order_date] ✗ ↑ pre-CLS schema used for SET * expansion mismatch with CLS-restricted FROM output → corrupted column assignments in MERGE
```

Fix

The CLS star expansion logic was extended to cover SET * and INSERT * in DeltaMerge plan nodes. Specifically, when a DeltaMergeIntoMatchedUpdateClause or DeltaMergeIntoNotMatchedInsertClause contains a wildcard assignment, the expansion is intercepted and resolved against the CLS-restricted source output — consistent with how SELECT * is handled in FROM clauses.

```
Before fix: SET * → expands against raw source schema → includes restricted columns After fix: SET * → expands against CLS secure projection output → only permitted columns
```

Test Case Added

The following test case was added to the regression suite to prevent recurrence:

```
-- Source is a CLS-restricted view: permitted cols = [cls_customer_id, cls_order_date] -- Restricted cols = [order_id, amount] MERGE INTO target t USING cls_restricted_view s ON t.cls_customer_id = s.cls_customer_id WHEN MATCHED THEN UPDATE SET * -- must expand to permitted cols only WHEN NOT MATCHED THEN INSERT * -- must expand to permitted cols only
```

Expected behaviour: SET * and INSERT * expand to only [cls_customer_id, cls_order_date] — matching the CLS-enforced source projection. Any attempt to reference restricted columns via wildcard expansion is blocked.

Principle Established

> *Wildcard (*) expansion in all syntactic positions — FROM, SELECT, SET, INSERT — must be resolved against the CLS-restricted output, not the raw relation schema. Any query plan node that performs its own internal star expansion must be explicitly covered by the CLS interception layer.*

Incident Report 3: Query Compiler Error Masking — Production Rollback

Incident Summary

A critical regression was introduced in the CLS framework that caused some queries to fail unexpectedly, including queries that were fully permitted and expected to succeed. The failure was non-deterministic — it affected a subset of queries depending on their plan shape and how deeply planLater was invoked during physical planning. This instability resulted in a production rollback. The root cause was the incorrect placement of an AnalysisException masking rule within the Spark query compilation pipeline, combined with a fundamental misunderstanding of how Spark's analysis lifecycle interacts with optimization and physical planning.

Background

Spark's AnalysisException stack traces can leak sensitive information. When CLS denies access to a restricted column, the resulting exception may contain:

- Internal column names from the underlying schema
- Attribute exprId references that expose the full relation schema
- Plan fragments revealing the existence of restricted columns

To prevent this, a masking rule was introduced late in the development cycle to intercept AnalysisException and replace the stack trace with a sanitised, user-facing error message before it reached the caller.

Root Cause

Incorrect Placement of the Masking Rule

The masking rule was injected as a resolution rule (injectResolutionRule) at the end of the resolver chain. The intent was to catch any AnalysisException thrown during analysis and sanitise it before it surfaced. However this placement was fundamentally wrong.

Spark's analysis is not a single linear pass. The query compilation lifecycle is:

```
Analysis (Analyzer.execute) ↓ Optimization (Optimizer.execute) ↓ Physical Planning
(SparkPlanner.plan) ■■■ planLater(logicalPlan) ← re-invokes analysis on sub-plans ■■■
planLater(logicalPlan) ← re-invokes analysis on sub-plans ■■■ ... ↓ Execution
(QueryExecution.executedPlan)
```

The critical detail is planLater — Spark's physical planner defers planning of sub-plans by calling planLater, which re-enters the analysis and planning pipeline recursively. This means analysis is invoked not just once at the beginning, but repeatedly from within optimization and physical planning. The masking rule, when it fired during a planLater-triggered re-analysis, was intercepting legitimate intermediate AnalysisExceptions that Spark uses internally as control flow signals — not user-facing errors.

```
Physical Planner: planLater(subPlan) → re-enters Analyzer → masking rule fires → intercepts
internal AnalysisException → replaces with sanitised error → Spark's internal recovery logic never
sees original exception → planning fails for affected queries ✗
```

Why Permitted Queries Failed

The failure was plan-shape dependent. Queries whose physical planning involved planLater on sub-plans that triggered internal AnalysisException handling were affected. Spark internally uses AnalysisException during planLater re-analysis as part of normal plan exploration — trying multiple planning strategies and falling back when one fails. The masking rule intercepted these internal exceptions for affected query shapes, breaking

Spark's own planning fallback mechanism regardless of whether CLS was involved. Simpler queries whose physical plans did not exercise this re-analysis path continued to work correctly — which is why the failure appeared non-deterministic and was not caught in pre-release testing.

The Realisation

> *AnalysisException masking cannot be safely placed anywhere in the Spark extension API — resolution rules, postHoc rules, or optimizer rules — because analysis is a back-and-forth process invoked from optimization and physical planning via planLater. Any rule that intercepts exceptions at the extension layer will interfere with Spark's internal planning control flow for queries whose plan shape exercises that re-entry path. The only safe place to mask AnalysisException is inside Spark's own source code — at the outermost QueryExecution boundary where the exception surfaces to the user, after all internal analysis, optimization, and planning re-entries are complete.*

```
QueryExecution.assertAnalyzed() ← safe interception point QueryExecution.executedPlan ← safe interception point ↑ only fires once, at the boundary between Spark internals and user-facing output no planLater re-entry possible at this point
```

Fix

The AnalysisException masking was removed from the Spark extension entirely and reimplemented as a patch to Spark's source code at the QueryExecution boundary:

```
// In QueryExecution.scala — patched Spark source def assertAnalyzed(): Unit = { try { analyzedPlan } catch { case e: AnalysisException => // Sanitise before surfacing to user // Strip internal plan fragments, exprIds, restricted column names throw CLSExceptionMasker.sanitise(e) } }
```

This ensures masking fires exactly once at the user-facing boundary, internal planLater re-analysis exceptions are never intercepted, Spark's planning fallback mechanism is fully preserved, and permitted queries are completely unaffected regardless of plan shape.

Timeline

Stage	Action
Late development	AnalysisException masking added as resolution rule
Release	Subset of queries begin failing in production — non-deterministic
Immediate response	Production rollback initiated
Root cause analysis	planLater re-analysis interference identified
Fix	Masking moved to QueryExecution boundary in patched Spark source
Re-release	Stable — only CLS-denied queries produce sanitised errors

Principle Established

> *Spark's analysis pipeline is not linear. Analysis, optimization, and physical planning are mutually recursive via planLater. Any cross-cutting concern such as exception masking, auditing, or telemetry that must fire exactly once at the user-facing boundary must be implemented at the QueryExecution level in Spark source — not as a Spark extension rule, which fires at every re-entry point in the recursive pipeline.*